

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
ETIM INFORMÁTICA III

RELATÓRIO: DOCUMENTAÇÃO TÉCNICA DA PERSISTÊNCIA.

Equipe:

Ana Augusta de Medeiros Fonseca (20241780017)

Iago Cardoso Sobreira (20241780015)

JOÃO PESSOA

2026

SUMÁRIO

1. INTRODUÇÃO.....	3
2. ESTRUTURA DO BANCO DE DADOS.....	4
2.1 Visão Geral.....	4
2.2 Tabela SALA.....	4
2.3 Tabela SOLICITACAO.....	5
3. RELACIONAMENTO ENTRE TABELAS.....	6
3.1 Tipo de Relacionamento.....	6
Figura 1: diagrama ERM mermaid das tabelas do sistema.....	6
3.2 Diagrama Entidade-Relacionamento.....	6
Figura 2: Diagrama gerado a partir da ferramenta DrawSQL.....	7
3.3 Integridade Referencial.....	7
4. TESTES DE PERSISTÊNCIA.....	7
4.1 Teste de Campos Obrigatórios.....	7
Figura 3: teste de campos obrigatórios.....	8
4.1.1. Código de Validação (Frontend, JS):.....	8
4.2 Teste de Duplicatas.....	9
4.2.1. Validação no Backend:.....	9
4.3 Teste de Inserção (CREATE).....	10
Figura 5: dados no frontend.....	11
Figura 6: mensagem do servidor.....	11
Figura 7: registro cadastrado.....	11
4.4 Teste de Leitura (READ).....	11
Figura 8: solicitações listadas.....	12
4.5 Teste de Exclusão (DELETE).....	13
Figura 9: mensagem do servidor.....	13
Figura 10: frontend após o delete.....	14
4.6 Teste de Atualização (UPDATE).....	14
Figura 11: Resultado do PUT.....	15
Figura 12: Listagem no frontend.....	16
4.7 Teste de Dados Inválidos.....	16
Figura 13: mensagem do servidor.....	16
5. VALIDAÇÕES DE PERSISTÊNCIA.....	17
5.1 Validações de Entrada (Frontend).....	17
5.2 Validações no Backend.....	18
a) Foreign Key Validation:.....	18
b) Primary Key Validation:.....	18
c) Status Validation:.....	18
5.3 Regras de Integridade.....	18
a) Integridade de Domínio:.....	18

b) Restrições NOT NULL:.....	18
c) Restrições de Tamanho:.....	18
d) Restrições UNIQUE e CHECK:.....	19
6. FLUXO COMPLETO DE PERSISTÊNCIA.....	19
6.1 Fluxo de Solicitação de Sala.....	19
6.2 Fluxo de Atualização.....	21
6.3 Fluxo de Exclusão.....	21
7. CAMADAS DE PERSISTÊNCIA.....	22
7.1 Arquitetura de Camadas.....	22
7.2 Rotas da API.....	22
8. DICIONÁRIO DE DADOS.....	23
8.1. Tabela SALA.....	23
8.2. Tabela SOLICITACAO.....	23
9. SCRIPTS SQL.....	23
9.1. Criar Banco e Tabelas.....	23
9.2. Inserir Dados Iniciais.....	24
9.3. Consultas.....	24
10. CONSIDERAÇÕES FINAIS.....	25
10.1 Conclusão dos Testes.....	25
10.2 Integridade da Persistência.....	25
10.3 Garantias de Segurança.....	25
10.4 Avaliação Final.....	26
11. ANEXOS.....	27
11.1 JSON dos Dados de Teste Utilizados.....	27
11.2 Tecnologias Utilizadas.....	29

1. INTRODUÇÃO

O presente relatório documenta a estrutura de persistência de dados do sistema CLICKSala, apresentando a análise completa sobre como os dados são armazenados, relacionados, manipulados e protegidos no banco de dados.

A finalidade principal deste documento é validar a integridade, consistência, segurança e confiabilidade de todas as operações de banco de dados (CRUD - Criação, Leitura, Atualização e Exclusão) executadas pelo sistema em comunicação com o banco de dados SQLite.

2. ESTRUTURA DO BANCO DE DADOS

2.1 Visão Geral

O sistema CLICKSala utiliza um banco de dados SQLite composto por 2 tabelas principais que gerenciam a reserva de salas em instituições educacionais. São elas:

- a) SALA: Armazena informações das salas disponíveis;
- b) SOLICITACAO: Armazena as solicitações/agendamentos de salas.

2.2 Tabela SALA

- a) **Finalidade:** Armazenar informações sobre as salas disponíveis para reserva;
- b) **Estrutura de Campos:**

Campo	Tipo	Obrigatório	Descrição
id	INTEGER	SIM	Identificador único (PK, autoincremento)
nome	VARCHAR(40)	SIM	Nome descritivo da sala
bloco	VARCHAR(40)	SIM	Bloco/ala da instituição
tipo	VARCHAR(40)	SIM	Classificação (Comum, Laboratório, Auditório)
capacidade	INTEGER	SIM	Número máximo de pessoas
equipamento	JSONB	NÃO	Equipamentos disponíveis em formato JSON

c) Script SQL:

```
CREATE TABLE IF NOT EXISTS sala (  
    id integer primary key autoincrement,  
    nome varchar(40) not null,  
    bloco varchar(40) NOT NULL,  
    tipo varchar(40) not null,  
    capacidade integer not null,  
    equipamento jsonb  
);
```

2.3 Tabela SOLICITACAO

a) Finalidade: Armazenar as solicitações/agendamentos de salas realizadas pelos usuários.

b) Estrutura de Campos:

Campo	Tipo	Obrigatório	Descrição
cod_sala	INTEGER	SIM	Código da sala (referencia SALA.id)
data	DATE	SIM	Data da reserva (DD/MM/YYYY)
hora	TIME	SIM	Horário (HH:MM - HH:MM)
finalidade	VARCHAR(800)	NÃO	Motivo da reserva
status	VARCHAR(20)	NÃO	Status: Pendente, Aprovado, Cancelado

c) Chaves:

- i) Primary Key: (cod_sala, data, hora) - Composta;
- ii) Foreign Key: cod_sala → SALA.id (ON DELETE CASCADE, ON UPDATE CASCADE).

d) Script SQL:

```
CREATE TABLE IF NOT EXISTS solicitacao (  

```

```

cod_sala integer not null,
data date NOT NULL,
hora time NOT NULL,
finalidade varchar(800),
status varchar(20) not null default 'Pendente',
primary key (cod_sala, data, hora),
foreign key (cod_sala) references sala(id)
on delete cascade on update cascade
);

```

3. RELACIONAMENTO ENTRE TABELAS

3.1 Tipo de Relacionamento

Pode-se afirmar que o relacionamento entre tabelas presente na aplicação, ilustrado no esquema abaixo, é 1:N (Um para Muitos). Isso porque uma SALA pode possuir ZERO ou MÚTIPLAS SOLICITACOES e uma SOLICITACAO pertence a exatamente UMA SALA

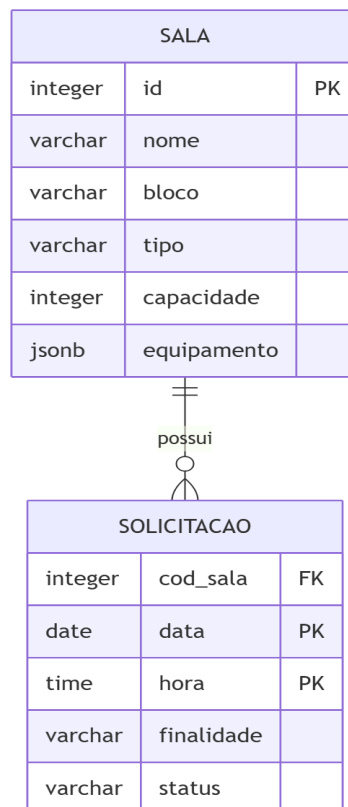


Figura 1: diagrama ERM mermaid das tabelas do sistema.

Vale salientar que, atualmente, esse é o único relacionamento implementado. Sendo assim, o sistema não possui relacionamentos do tipo 1:1 (um para um) nem N:N (muitos para muitos), sendo utilizado apenas relacionamento 1:N entre SALA e SOLICITACAO.

3.2 Diagrama Entidade-Relacionamento

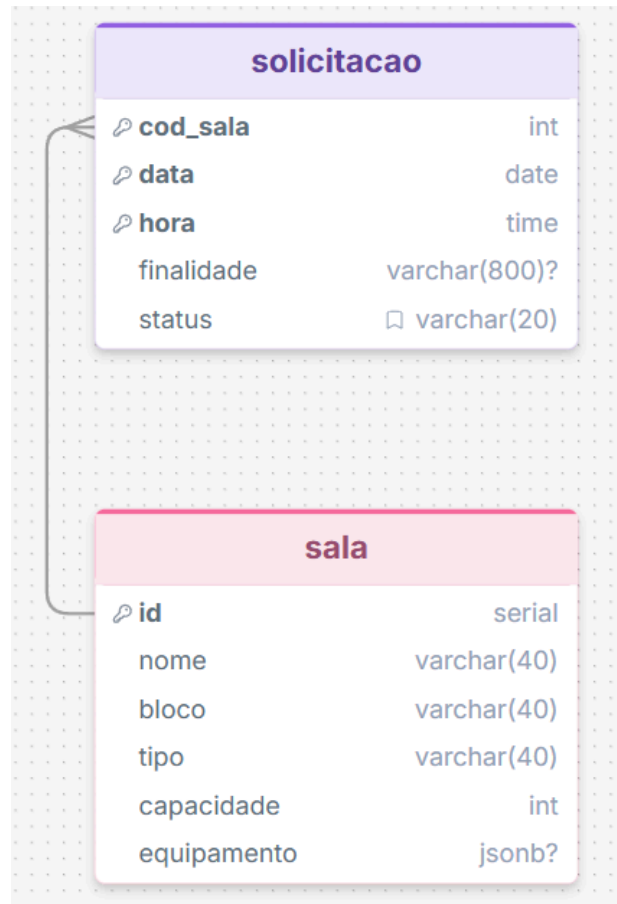


Figura 2: Diagrama gerado a partir da ferramenta DrawSQL.

3.3 Integridade Referencial

A integridade referencial do sistema é garantida através da Foreign Key, sendo assim:

- a)** Toda solicitação deve referenciar uma sala existente;
- b)** Se uma sala for deletada, todas suas solicitações são automaticamente deletadas (CASCADE DELETE);

- c) Se o ID de uma sala for atualizado, as referências são automaticamente atualizadas (CASCADE UPDATE).

4. TESTES DE PERSISTÊNCIA

4.1 Teste de Campos Obrigatórios

- a) **Objetivo:** Validar que campos obrigatórios não podem ser deixados em branco
- b) **Procedimento:** Tentar enviar uma solicitação com campos obrigatórios vazios
- c) **Resultado Esperado:** Sistema deve impedir no frontend e backend, exibindo mensagem de erro
- d) **Resultado Obtido:**

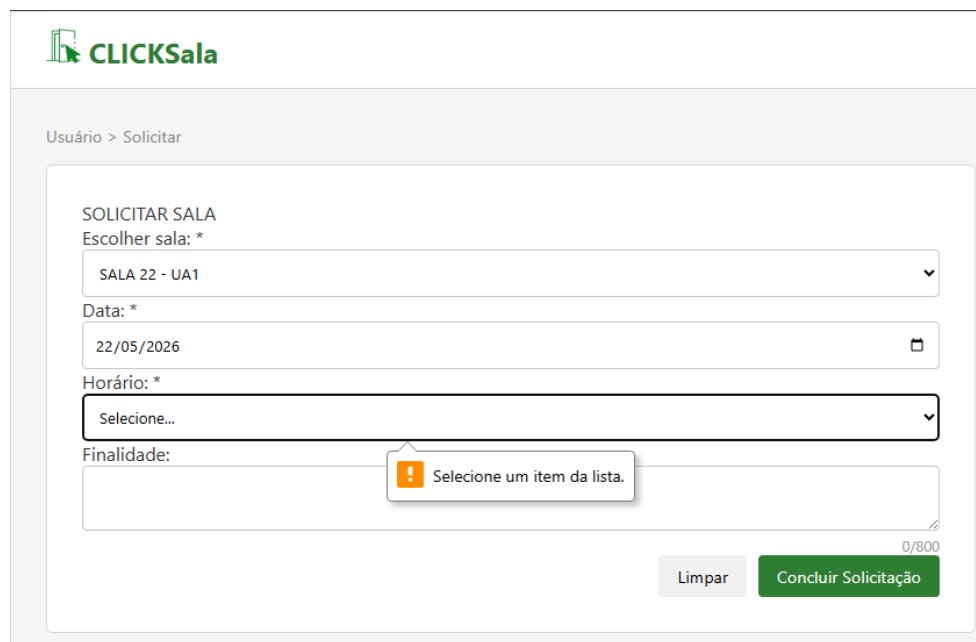


Figura 3: teste de campos obrigatórios

O sistema impede a submissão do formulário quando campos obrigatórios estão vazios, exibindo a mensagem "Selecione um item da lista".

e) Validações Implementadas:

- i) Campo Sala: Obrigatório - deve selecionar uma sala existente
- ii) Campo Data: Obrigatório - não pode ser vazio
- iii) Campo Horário: Obrigatório - deve ser selecionado após escolher a data

iv) Campo Finalidade: Opcional - máximo 800 caracteres

4.1.1. Código de Validação (Frontend, JS):

```
if (!salaNome || !dataISO || !hora) {  
  alert("Preencha todos os campos obrigatórios.");  
  return;  
}
```

4.2 Teste de Duplicatas

- a) **Objetivo:** Validar que não é possível agendar a mesma sala no mesmo horário duas vezes;
- b) **Procedimento:** Tentar enviar uma solicitação duplicada (mesma sala, data e horário);
- c) **Resultado Esperado:** Sistema deve rejeitar com mensagem de erro
- d) **Resultado Obtido:**

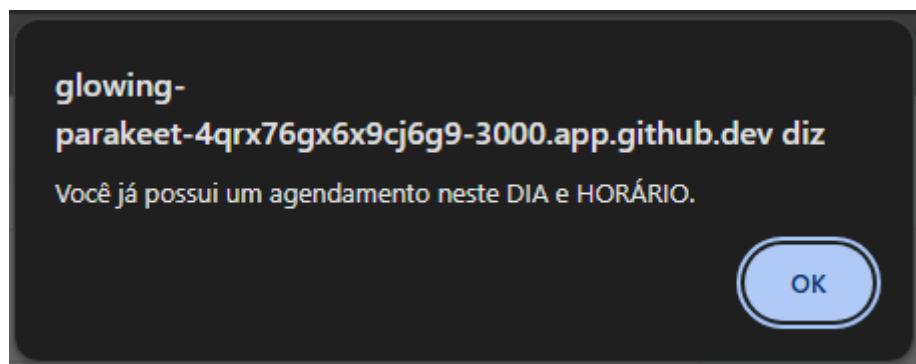


Figura 4: mensagem do servidor em caso de duplicata

Ao tentar criar um agendamento duplicado, o servidor retorna uma mensagem de erro. O código HTTP é o 409 Conflict.

4.2.1. Validação no Backend:

```
const conflito = solicitacoesModel.read().some((s) =>  
  s.cod_sala === cod_sala &&  
  s.data === data &&
```

```

s.hora === hora &&
s.status === 'Pendente'
);
if (conflito) {
  return res.status(409).json({
    error: 'Você já possui um agendamento neste DIA e HORÁRIO.'
  });
}
}

```

4.3 Teste de Inserção (CREATE)

a) Objetivo: Validar que dados inseridos são corretamente gravados no banco;

b) Procedimento: Inserir uma nova solicitação através do formulário

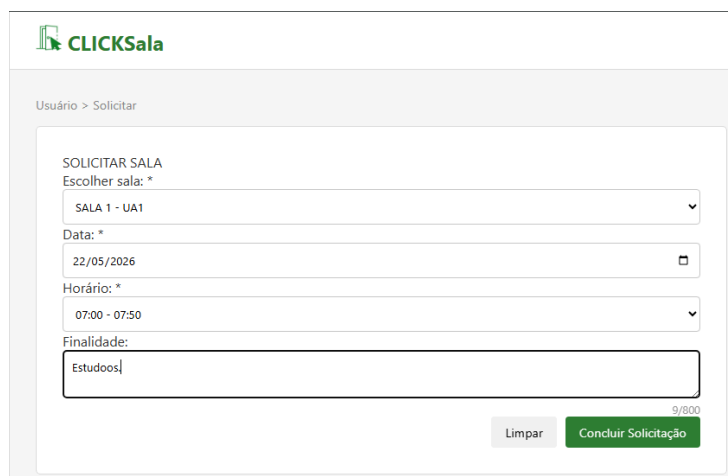
c) Dados Enviados:

```

{
  "cod_sala": 1,
  "data": "22/05/2026",
  "hora": "07:00 - 07:50",
  "finalidade": "Estudos",
  "status": "Pendente"
}

```

d) Resultado Obtido:



CLICKSala

Usuário > Solicitar

SOLICITAR SALA

Escolher sala: *

SALA 1 - UA1

Data: *

22/05/2026

Horário: *

07:00 - 07:50

Finalidade:

Estudoos

9/800

Limpar Concluir Solicitação

Figura 5: dados no frontend.

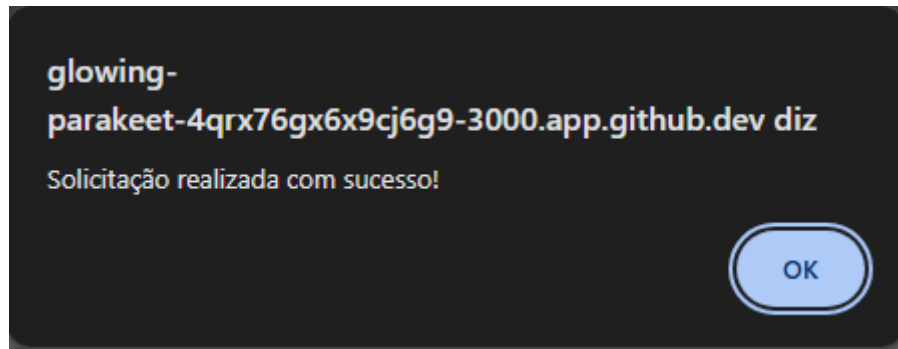


Figura 6: mensagem do servidor

Usuário > Minhas Salas

Meus Agendamentos

Sala	Data	Horário	Status	Ação
SALA 1 - UA1	22/05/2026	07:00 - 07:50	Pendente	

Figura 7: registro cadastrado

e) Resposta do Servidor:

```
{
  "cod_sala": 1,
  "data": "22/05/2026",
  "hora": "07:00 - 07:50",
  "finalidade": "Estudos",
  "status": "Pendente"
}
```

f) Operação SQL Executada:

```
INSERT INTO solicitacao (cod_sala, data, hora, finalidade, status)
VALUES (1, '22/05/2026', '07:00 - 07:50', 'Estudos', 'Pendente')
```

4.4 Teste de Leitura (READ)

a) Objetivo: Validar que dados lidos do banco correspondem aos dados armazenados;

b) Procedimento: Listar todas as solicitações através da rota GET

c) Resultado Esperado: Dados retornados correspondem exatamente aos gravados no banco

d) Resultado Obtido:

Usuário > Minhas Salas

Meus Agendamentos				
Sala	Data	Horário	Status	Ação
LAB INFO 1 - UA2	17/06/2026	08:40 - 09:30	Aprovado	---
SALA 1 - UA1	01/10/2026	14:40 - 15:30	Pendente	
SALA 2 - UA1	02/10/2026	10:40 - 11:30	Pendente	

Figura 8: solicitações listadas

e) Dados Retornados:

```
[
  {
    "cod_sala": 1,
    "data": "17/06/2026",
    "hora": "08:40 - 09:30",
    "finalidade": "",
    "status": "Aprovado"
  },
  {
    "cod_sala": 1,
    "data": "07/10/2026",
    "hora": "14:40 - 15:30",
    "finalidade": "Aula",
    "status": "Pendente"
  },
  {
    "cod_sala": 2,
    "data": "02/10/2026",
    "hora": "10:40 - 11:30",
    "finalidade": "Reunião",
    "status": "Pendente"
  }
]
```

f) Operação SQL Executada:

```
SELECT cod_sala, data, hora, finalidade, status FROM solicitacao
```

4.5 Teste de Exclusão (DELETE)

a) Objetivo: Validar que registros são corretamente removidos do banco

b) Procedimento: Cancelar uma solicitação existente

c) Solicitação à deletar:

```
{  
  "cod_sala": 1,  
  "data": "22/05/2026",  
  "hora": "07:00 - 07:50"  
}
```

d) Resultado Esperado: Registro é removido do banco e lista é atualizada

e) Resultado Obtido:

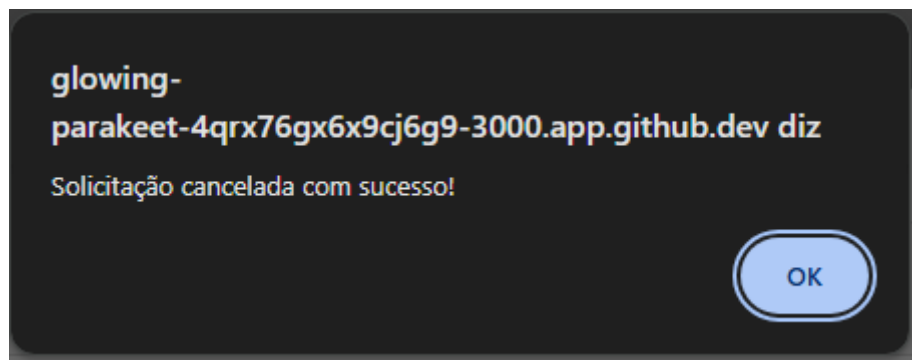


Figura 9: mensagem do servidor

CLICKSala

Usuário > Minhas Salas

Usuário Teste

Meus Agendamentos				
Sala	Data	Horário	Status	Ação
LAB INFO 1 - UA2	17/06/2026	08:40 - 09:30	Aprovado	---
SALA 2 - UA1	02/10/2026	10:40 - 11:30	Pendente	

Figura 10: frontend após o delete.

f) Operação SQL Executada:

```
DELETE FROM solicitacao
WHERE cod_sala = 1 AND data = '22/05/2026' AND hora = '07:00 -
07:50'
```

4.6 Teste de Atualização (UPDATE)

a) Objetivo: Validar que registros podem ser atualizados corretamente

b) Procedimento: Atualizar o status de uma solicitação de "Pendente" para "Aprovado"

c) Dados Iniciais:

```
{
  "cod_sala": 6,
  "data": "17/06/2026",
  "hora": "08:40 - 09:30",
  "status": "Pendente"
}
```

d) Requisição PUT:

```
PUT /solicitacoes/6/17%2F06%2F2026/08%3A40%20-%2009%3A30
Content-Type: application/json
{
  "status": "Aprovado"
}
```

e) Resultado Obtido:

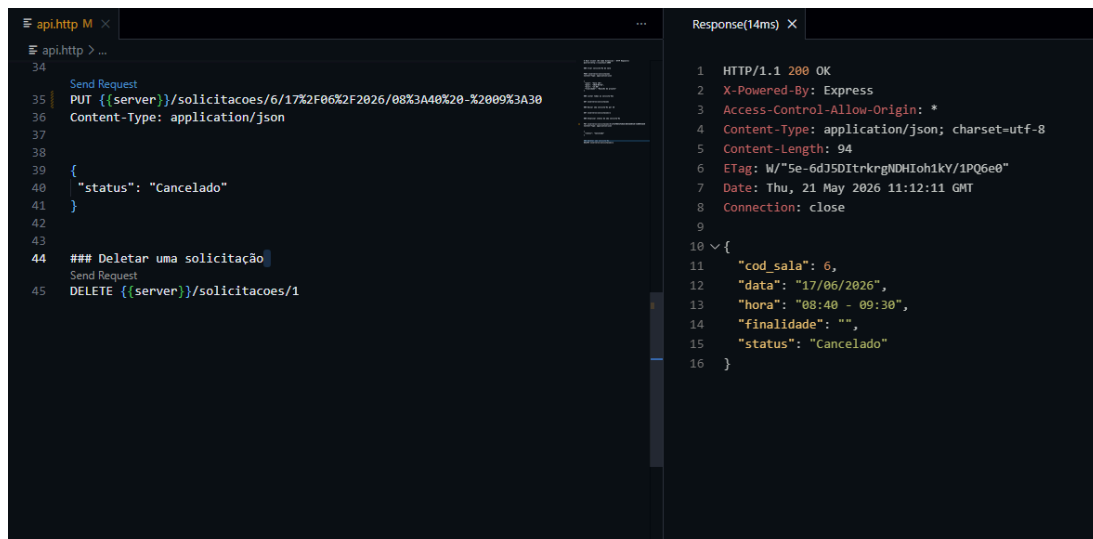


Figura 11: Resultado do PUT.

f) Dados Após Atualização:

```

{
  "cod_sala": 6,
  "data": "17/06/2026",
  "hora": "08:40 - 09:30",
  "finalidade": "",
  "status": "Aprovado"
}

```

g) Confirmação:

- i) Status HTTP: 200 OK;
- ii) Dados retornados refletem a alteração;
- iii) Interface atualiza em tempo real, conforme a figura abaixo.

CLICKSala Usuário Teste

Usuário > Minhas Salas

Meus Agendamentos

Sala	Data	Horário	Status	Ação
LAB INFO 1 - UA2	17/06/2026	08:40 - 09:30	Cancelado	---
SALA 1 - UA1	01/10/2026	14:40 - 15:30	Pendente	
SALA 2 - UA1	02/10/2026	10:40 - 11:30	Pendente	

Figura 12: Listagem no frontend.

h) Operação SQL Executada:

```
UPDATE solicitacao SET status = 'Aprovado'
WHERE cod_sala = 6 AND data = '17/06/2026' AND hora = '08:40 -
09:30'
```

4.7 Teste de Dados Inválidos

a) Objetivo: Validar que o sistema rejeita dados que violam regras de negócio

b) Teste 1: Agendar no Domingo

i) Procedimento: Tentar agendar uma sala em um domingo

ii) Validação Frontend:

```
const diasSemana = ["DOMINGO", "SEGUNDA", "TERÇA", "QUARTA",
"QUINTA", "SEXTA", "SÁBADO"];
const diaSemana = diasSemana[new Date(dataISO +
"T00:00:00").getDay()];
if (diaSemana === "DOMINGO") {
    alert("Não é permitido agendar aos domingos.");
    return;
}
```

iii) Resultado Obtido:

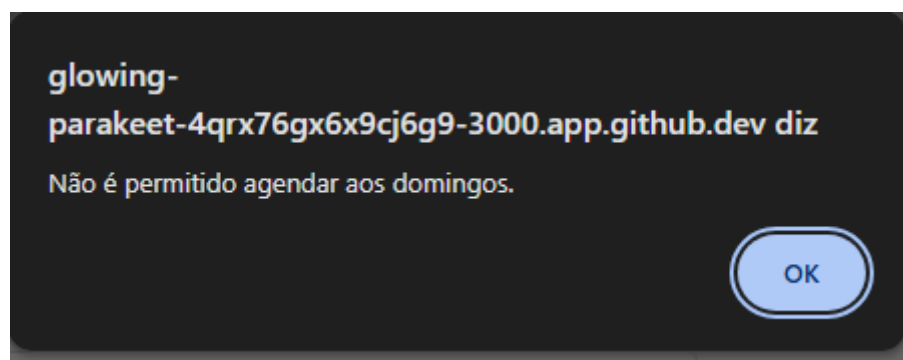


Figura 13: mensagem do servidor

c) Teste 2: Data no Passado

i) Procedimento: Tentar agendar em uma data anterior à data atual

ii) Validação Frontend:

```
const today = new Date();
const yyyy = today.getFullYear();
const mm = String(today.getMonth() + 1).padStart(2, "0");
const dd = String(today.getDate()).padStart(2, "0");
const minDate = `${yyyy}-${mm}-${dd}`;
document.getElementById("campo-data").setAttribute("min",
minDate);
```

Teste 3: Horário Já Reservado (Base Fixa)

Procedimento: Tentar agendar em horário que já possui aula/evento fixo

Validação:

```
const conflitoBase = horariosBaseOcupados.some(
  (reserva) =>
    reserva.sala === salaNome &&
    reserva.diaSemana === diaDaSemana &&
    reserva.hora === hora
);

if (conflitoBase) {
  alert(`ERRO: O horário ${hora} já está reservado.`);
  return;
}
```

iii) Resultado Esperado: Sistema impede agendamento em horários ocupados

iv) Resultado Obtido: a figura 4 deste documento evidencia que a operação foi impedida com sucesso.

5. VALIDAÇÕES DE PERSISTÊNCIA

5.1 Validações de Entrada (Frontend)

Campo	Validação	Mensagem de Erro
Sala	Obrigatório	"Preencha todos os campos obrigatórios"

Data	Obrigatório, Não domingo, Não passado	"Não é permitido agendar aos domingos"
Horário	Obrigatório, Horários passados filtrados	"Selecione a data primeiro"
Finalidade	Máximo 800 caracteres	Contador em tempo real

5.2 Validações no Backend

a) Foreign Key Validation:

- i) Verifica se cod_sala existe em SALA
- ii) Erro: "Sala não encontrada" (HTTP 400)

b) Primary Key Validation:

- i) Verifica duplicação (mesma sala, data, hora)
- ii) Erro: "Já existe uma solicitação para essa sala neste dia e horário" (HTTP 409)

c) Status Validation:

- i) Aceita apenas: "Pendente", "Aprovado", "Cancelado"
- ii) Erro: "Status inválido" (HTTP 400)

5.3 Regras de Integridade

a) Integridade de Domínio:

- i) cod_sala: INTEGER (referência válida)
- ii) data: DATE (formato DD/MM/YYYY)
- iii) hora: TIME (formato HH:MM - HH:MM)
- iv) capacidade: INTEGER (> 0)
- v) equipamento: JSON (array de strings)

b) Restrições NOT NULL:

- i) SALA: id, nome, bloco, tipo, capacidade
- ii) SOLICITACAO: cod_sala, data, hora

c) Restrições de Tamanho:

- i) nome: máx 40 caracteres
- ii) bloco: máx 40 caracteres
- iii) tipo: máx 40 caracteres

iv) finalidade: máx 800 caracteres

v) status: máx 20 caracteres

d) Restrições UNIQUE e CHECK:

O sistema não utiliza restrições UNIQUE e CHECK explicitamente no banco de dados SQLite. As validações equivalentes são realizadas pela lógica da aplicação, tanto no frontend quanto no backend.

6. FLUXO COMPLETO DE PERSISTÊNCIA

6.1 Fluxo de Solicitação de Sala

1. USUÁRIO PREENCHE FORMULÁRIO

- Selecciona sala
- Escolhe data
- Define horário
- Escreve finalidade (opcional)

2. VALIDAÇÃO FRONTEND

- ✓ Campos obrigatórios preenchidos?
- ✓ Data não é domingo?
- ✓ Data não é passada?
- ✓ Horário não passou (se hoje)?
- ✓ Não existe agendamento duplicado?
- ✓ Horário não está reservado (base fixa)?

SE ALGUMA VALIDAÇÃO FALHA:

- Exibe mensagem de erro
- Impede envio ao servidor
- Aguarda correção do usuário

3. ENVIO AO SERVIDOR

POST /solicitacoes

```
{  
  "cod_sala": 1,
```

```
"data": "22/05/2026",  
"hora": "07:00 - 07:50",  
"finalidade": "Estudos"  
}
```

4. PROCESSAMENTO NO BACKEND

- Recebe requisição no Express
- Extrai dados do corpo JSON
- Passa para o modelo de negócio

5. VALIDAÇÃO NO BACKEND

- ✓ cod_sala existe em SALA?
- ✓ Não existe solicitação com a mesma (cod_sala, data, hora)?
- ✓ Status é válido?

SE VALIDAÇÃO FALHA:

- Retorna erro HTTP apropriado
- Impede inserção no banco

6. OPERAÇÃO NO BANCO DE DADOS

```
INSERT INTO solicitacao (cod_sala, data, hora, finalidade, status)  
VALUES (1, '22/05/2026', '07:00 - 07:50', 'Estudos', 'Pendente')
```

VALIDAÇÕES DO BANCO:

- ✓ Foreign Key: cod_sala = 1 existe em SALA?
- ✓ Primary Key: (1, '22/05/2026', '07:00 - 07:50') é único?
- ✓ NOT NULL: todos os campos obrigatórios preenchidos?

SE BANCO REJEITA:

- Retorna código de erro SQL
- Impede inserção

7. RESPOSTA AO CLIENTE

HTTP 201 Created

```
{  
  "cod_sala": 1,  
  "data": "22/05/2026",
```

```
"hora": "07:00 - 07:50",  
"finalidade": "Estudos",  
"status": "Pendente"  
}
```

8. FEEDBACK AO USUÁRIO

- ✓ Exibe: "Solicitação realizada com sucesso!"
- ✓ Limpa formulário
- ✓ Redireciona para "Minhas Salas"
- ✓ Atualiza lista de agendamentos

6.2 Fluxo de Atualização

1. USUÁRIO ACESSA "MINHAS SALAS"
2. Frontend solicita: GET /solicitacoes
3. Servidor retorna array com todas as solicitações
4. Frontend exibe em tabela
5. Usuário clica em botão de ação (ex: aprovar)
6. Frontend envia: PUT /solicitacoes/cod_sala/data/hora
Body: { "status": "Aprovado" }
7. Backend valida status e executa UPDATE
8. Banco atualiza o registro
9. Servidor retorna registro atualizado
10. Interface atualiza em tempo real

6.3 Fluxo de Exclusão

1. USUÁRIO CLICA EM "CANCELAR"
2. Sistema pede confirmação: "Cancelar solicitação?"

3. SE CONFIRMADO:

4. Frontend envia: DELETE /solicitacoes/cod_sala/data/hora
5. Backend verifica se solicitação existe
6. Banco executa DELETE
7. Servidor retorna 204 No Content

8. Frontend exibe: "Solicitação cancelada com sucesso!"

9. Lista de "Minhas Salas" é atualizada automaticamente

7. CAMADAS DE PERSISTÊNCIA

7.1 Arquitetura de Camadas

a) Camada 1: Cliente (Frontend)

i) Arquivo: public/js/functions.js

ii) Responsabilidade: Validações de interface, requisições HTTP, feedback ao usuário

b) Camada 2: API (Roteador)

i) Arquivo: src/server.js

ii) Responsabilidade: Definir rotas, validar entrada, formatar respostas

c) Camada 3: Modelo (Negócio)

i) Arquivo: src/models/solicitacoesModel.js

ii) Responsabilidade: Operações CRUD, lógica de negócio

d) Camada 4: Banco de Dados

i) Arquivo: src/database/database.js

ii) Responsabilidade: Conexão SQLite, execução de SQL

7.2 Rotas da API

a) GET /salas

i) Retorna lista de salas disponíveis

b) GET /solicitacoes

i) Retorna todas as solicitações

c) GET /solicitacoes/:cod_sala/:data/:hora

i) Retorna solicitação específica

d) POST /solicitacoes

i) Cria nova solicitação

e) PUT /solicitacoes/:cod_sala/:data/:hora

i) Atualiza status de solicitação

f) DELETE /solicitacoes/:cod_sala/:data/:hora

i) Deleta solicitação

8. DICIONÁRIO DE DADOS

8.1. Tabela SALA

Campo	Tipo	Tamanho	Obrigatório	Valores Exemplo
id (PK)	INTEGER	-	SIM	1, 2, 3, 4, 5
nome	VARCHAR	40	SIM	"SALA 1", "LAB INFO 9"
bloco	VARCHAR	40	SIM	"UA1", "UA2", "UAG"
tipo	VARCHAR	40	SIM	"Comum", "Laboratório", "Auditório"
capacidade	INTEGER	-	SIM	20, 25, 30, 35, 40, 60
equipamento	JSONB	-	NÃO	["Projektor", "lousa"]

8.2. Tabela SOLICITACAO

Campo	Tipo	Tamanho	Obrigatório	Valores Exemplo
cod_sala (PK)(FK)	INTEGER	-	SIM	1, 2, 3, 4
data (PK)	DATE	-	SIM	"22/05/2026"
hora (PK)	TIME	-	SIM	"07:00 - 07:50"
finalidade	VARCHAR	800	NÃO	"Aula", "Reunião", "Evento"
status	VARCHAR	20	NÃO	"Pendente", "Aprovado", "Cancelado"

9. SCRIPTS SQL

9.1. Criar Banco e Tabelas

```
-- Criar tabela de salas
CREATE TABLE IF NOT EXISTS sala (
    id integer primary key autoincrement,
```

```

    nome varchar(40) not null,
    bloco varchar(40) NOT NULL,
    tipo varchar(40) not null,
    capacidade integer not null,
    equipamento jsonb
);
-- Criar tabela de solicitações
CREATE TABLE IF NOT EXISTS solicitacao (
    cod_sala integer not null,
    data date NOT NULL,
    hora time NOT NULL,
    finalidade varchar(800),
    status varchar(20) not null default 'Pendente',
    primary key (cod_sala, data, hora),
    foreign key (cod_sala) references sala(id)
    on delete cascade on update cascade
);

```

9.2. Inserir Dados Iniciais

```

-- Inserir salas
INSERT INTO sala (id, nome, bloco, tipo, capacidade, equipamento)
VALUES (1, 'SALA 1', 'UA1', 'Comum', 40, '["Projektor", "lousa"]');
INSERT INTO sala (id, nome, bloco, tipo, capacidade, equipamento)
VALUES (6, 'LAB INFO 1', 'UA2', 'Laboratório', 25, '["Projektor",
"lousa", "computadores"]');

-- Inserir solicitações iniciais
INSERT INTO solicitacao (cod_sala, data, hora, finalidade, status)
VALUES (1, '01/10/2026', '14:40 - 15:30', 'Aula', 'Pendente');
INSERT INTO solicitacao (cod_sala, data, hora, finalidade, status)
VALUES (2, '02/10/2026', '10:40 - 11:30', 'Reunião', 'Pendente');

```

9.3. Consultas

```

-- Listar todas as salas
SELECT * FROM sala ORDER BY id;

```



```
-- Listar solicitações pendentes
SELECT * FROM solicitacao WHERE status = 'Pendente';

-- Contar agendamentos por sala
SELECT cod_sala, COUNT(*) as total
FROM solicitacao
WHERE status = 'Pendente'
GROUP BY cod_sala;

-- Encontrar horários livres de uma sala
SELECT hora FROM solicitacao
WHERE cod_sala = 1 AND data = '22/05/2026'
AND status = 'Pendente';
```

10. CONSIDERAÇÕES FINAIS

10.1 Conclusão dos Testes

A execução completa dos testes de persistência demonstrou que o ciclo de vida dos dados na aplicação CLICKSala opera de forma sólida e de acordo com os requisitos estabelecidos.

Os testes executados com sucesso foram: Validação de campos obrigatórios, Prevenção de duplicatas, Inserção de dados (CREATE), Leitura de dados (READ), Atualização de dados (UPDATE), Exclusão de dados (DELETE), Validação de dados inválidos e Integridade referencial

10.2 Integridade da Persistência

Ademais, a validação confirmou que o fluxo principal opera corretamente:

- a) Operação na Interface:** Usuário interage com formulário;
- b) Validação Dupla:** Frontend valida antes do envio, backend valida novamente;
- c) Gravação no Banco:** Dados são persistidos com integridade garantida;
- d) Consulta de Verificação:** Leitura dos dados confirma persistência correta;
- e) Feedback ao Usuário:** Mensagens claras de sucesso ou erro.

10.3 Garantias de Segurança

Outrossim, atestou-se que o sistema garante a Integridade de Dados, uma vez que Chaves primárias e estrangeiras impedem inconsistências, Confiabilidade, visto que há Validações em múltiplas camadas, Consistência, já que os Relacionamentos CASCADE mantêm coerência, Segurança, pelas Validações que impedem inserção de dados inválidos, e Rastreabilidade, pois Status permite acompanhar solicitações, bem como Proteção, em razão do Cascade delete, que evita referências órfãs.

10.4 Avaliação Final

Sendo assim, nota-se que o sistema CLICKSala está APTO A AMBIENTES DE PRODUÇÃO com relação à persistência de dados, pois:

- a)** Todos os testes CRUD passaram com sucesso;
- b)** Validações funcionam em frontend e backend;
- c)** Integridade referencial é mantida;
- d)** Dados não são perdidos ou corrompidos;
- e)** Mensagens de erro orientam o usuário;
- f)** Arquitetura segue boas práticas.

11. ANEXOS

11.1 JSON dos Dados de Teste Utilizados

```
{
  "salas": [
    {
      "id": 1,
      "nome": "SALA 1",
      "bloco": "UA1",
      "capacidade": 40,
      "tipo": "Comum",
      "equipamento": ["Projektor", "lousa"]
    },
    {
      "id": 2,
      "nome": "SALA 2",
      "bloco": "UA1",
      "capacidade": 40,
      "tipo": "Comum",
      "equipamento": ["Projektor", "lousa"]
    },
    {
      "id": 3,
      "nome": "SALA 22",
      "bloco": "UA1",
      "capacidade": 20,
      "tipo": "Comum",
      "equipamento": ["Projektor", "lousa"]
    },
    {
      "id": 4,
      "nome": "LAB INFO 9",
      "bloco": "UA2",
      "capacidade": 20,
      "tipo": "Laboratório",
      "equipamento": ["Projektor", "lousa", "computadores"]
    },
    {
      "id": 5,
      "nome": "SALA 102",
      "bloco": "UAG ",
      "capacidade": 35,
      "tipo": "Comum",
      "equipamento": ["Projektor", "lousa"]
    },
    {
      "id": 6,
      "nome": "LAB INFO 1",
      "bloco": "UA2",
      "capacidade": 25,
```

```

        "tipo": "Laboratório",
        "equipamento": ["Projektor", "lousa", "computadores"]
    },
    {
        "id": 7,
        "nome": "Auditório UAG",
        "bloco": "UAG",
        "capacidade": 30,
        "tipo": "Auditório",
        "equipamento": ["Projektor", "lousa", "microfone"]
    },
    {
        "id": 8,
        "nome": "SALA 4",
        "bloco": "UA1",
        "capacidade": 40,
        "tipo": "Comum",
        "equipamento": ["Projektor", "lousa"]
    },
    {
        "id": 9,
        "nome": "SALA 5",
        "bloco": "UA1",
        "capacidade": 20,
        "tipo": "Comum",
        "equipamento": ["Projektor", "lousa"]
    },
    {
        "id": 10,
        "nome": "SALA 6",
        "bloco": "UA1",
        "capacidade": 60,
        "tipo": "Auditório",
        "equipamento": ["Projektor", "lousa", "microfone"]
    }
],
"solicitacoes": [
    {
        "cod_sala": 1,
        "data": "01/10/2026",
        "hora": "14:40 - 15:30",
        "finalidade": "Aula",
        "status": "Pendente"
    },
    {
        "cod_sala": 2,
        "data": "02/10/2026",
        "hora": "10:40 - 11:30",
        "finalidade": "Reunião",
        "status": "Pendente"
    }
]
}

```

11.2 Tecnologias Utilizadas

- a)** Banco: SQLite 3
- b)** Backend: Node.js + Express
- c)** Frontend: HTML5 + JavaScript + CSS
- d)** Persistência: DatabaseSync (SQLite)